

Atelier — Construire une API Calculatrice en Node.js avec Tests et CI

Durée : 4h

Prérequis : Node.js installé (v22+), compte GitHub, bases de JavaScript

Présentation

Dans cet atelier, vous allez construire de zéro une API REST en Node.js **sans aucun framework**, l'accompagner d'une suite de tests complète avec **Jest**, puis automatiser ces tests avec un pipeline **GitHub Actions**.

À chaque étape, vous écrirez du code, l'exécuterez, et vérifierez que votre implémentation est correcte avant de passer à la suivante.

Étape 1 — Initialiser le projet

1.1 Créer le dépôt et la structure

Créez un nouveau dépôt public sur GitHub intitulé `calculator-api-js`, clonez-le, puis créez l'arborescence suivante :

```
calculator-api-js/  
├─ src/
```

```

|   └─ calculator.js      ← logique métier (étape 2)
|   └─ server.js         ← serveur HTTP (étape 4)
└─ tests/
    └─ calculator.test.js ← tests unitaires (étape
3)
    └─ api.test.js        ← tests d'intégration
(étape 6)
        └─ helpers/
            └─ http.js     ← utilitaire HTTP pour les
tests (étape 5)
└─ .github/
    └─ workflows/
        └─ ci.yml         ← pipeline CI (étape 7)
└─ eslint.config.js
└─ jest.config.js
└─ package.json

```

```

npm init -y
npm install --save-dev jest eslint @eslint/js

```

1.2 Configurer package.json

Ajoutez ces scripts dans package.json :

```

"scripts": {
  "start": "node src/server.js",
  "lint": "eslint src/ tests/",
  "test": "jest",
  "test:unit": "jest
tests/calculator.test.js",
  "test:integration": "jest tests/api.test.js",

```

```
"test:coverage": "jest --coverage",  
"test:ci": "jest --ci --coverage"  
}
```

Et déclarez les devDependencies :

```
"devDependencies": {  
  "@eslint/js": "^9.0.0",  
  "eslint": "^9.0.0",  
  "jest": "^29.7.0"  
}
```

1.3 Configurer Jest (jest.config.js)

```
module.exports = {  
  testEnvironment: "node",  
  coverageDirectory: "coverage",  
  collectCoverageFrom: ["src/**/*.js"],  
  coverageThreshold: {  
    global: {  
      branches: 90,  
      functions: 90,  
      lines: 90,  
      statements: 90,  
    },  
  },  
};
```

`coverageThreshold` force Jest à échouer si la couverture tombe sous 90%.

Cela vous oblige à écrire des tests sérieux, pas seulement les cas heureux.

1.4 Configurer ESLint (`eslint.config.js`)

```
const js = require("@eslint/js");

module.exports = [
  js.configs.recommended,
  {
    languageOptions: {
      ecmaVersion: 2022,
      globals: {
        require: "readonly",
        module: "readonly",
        exports: "readonly",
        process: "readonly",
        console: "readonly",
      },
    },
    rules: {
      "no-unused-vars": "warn",
      "no-console": "off",
    },
  },
];
```

Vérification

```
npm run lint # doit terminer sans erreur
```

Étape 2 — La logique métier :

`src/calculator.js`

Ce que vous devez produire

Une **classe** `Calculator` qui implémente les quatre opérations arithmétiques de base, sans aucune dépendance externe.

Spécification

Méthode	Signature	Comportement
Additionner	<code>add(a, b)</code>	Retourne <code>a + b</code>
Soustraire	<code>subtract(a, b)</code>	Retourne <code>a - b</code>
Multiplier	<code>multiply(a, b)</code>	Retourne <code>a * b</code>
Diviser	<code>divide(a, b)</code>	Retourne <code>a / b</code> — lève une <code>Error</code> si <code>b === 0</code>

Message d'erreur exact pour la division par zéro :

```
Division par zéro impossible.
```

La classe doit être exportée avec `module.exports`.

Exemple d'utilisation attendu

```
const calc = new Calculator();
calc.add(2, 3);           // 5
calc.subtract(10, 4);     // 6
calc.multiply(6, 7);      // 42
```

```
calc.divide(20, 5);    // 4
calc.divide(10, 0);    // → throw Error("Division par
zéro impossible.")
```

À vous de jouer

Écrivez la classe dans `src/calculator.js`. Vous validerez votre implémentation à l'étape suivante grâce aux tests.

Étape 3 — Tests unitaires : `tests/calculator.test.js`

Avant d'écrire le serveur, vous testez la logique pure **en isolation** :

aucun HTTP, aucun réseau, juste la classe `Calculator`.

Rappel : anatomie d'un test Jest

```
const Calculator = require("../src/calculator");

describe("Calculator", () => {
  let calculator;

  beforeEach(() => {
    calculator = new Calculator(); // instance
    fraîche avant chaque test
  });

  describe("add", () => {
    it("devrait retourner 5 pour add(2, 3)", () =>
    {
```

```

        expect(calculator.add(2, 3)).toBe(5);
    });
});
});

```

Matchers utiles

Matcher	Quand l'utiliser
<code>.toBe(valeur)</code>	Égalité stricte (<code>===</code>)
<code>.toBeCloseTo(valeur)</code>	Comparaison de flottants ($0.1 + 0.2 \approx 0.3$)
<code>.toThrow(erreur)</code>	Vérifie qu'une exception est levée
<code>.toBeNaN()</code>	Vérifie que la valeur est NaN

Astuce : `it.each` pour éviter la répétition

Plutôt que d'écrire un `it` par cas, utilisez une **table de données** :

```

it.each`
  a      | b      | expected
  ${2}   | ${3}   | ${5}
  ${-5}  | ${-3}  | ${-8}
  ${7}   | ${0}   | ${7}
`("devrait retourner $a + $b = $expected", ({ a, b,
expected }) => {
  expect(calculator.add(a, b)).toBe(expected);
});

```

Cas à couvrir pour chaque opération

add

a	b	Résultat	Matcher
2	3	5	toBe
-5	-3	-8	toBe
-5	3	-2	toBe
7	0	7	toBe
0.1	0.2	≈ 0.3	toBeCloseTo

subtract

a	b	Résultat	Matcher
10	4	6	toBe
3	10	-7	toBe
5	0	5	toBe
-5	-3	-2	toBe
0.3	0.1	≈ 0.2	toBeCloseTo

multiply

a	b	Résultat	Matcher
6	7	42	toBe
0	999	0	toBe
-3	-4	12	toBe
3	-4	-12	toBe
0.1	0.2	≈ 0.02	toBeCloseTo

divide

a	b	Résultat	Matcher
20	5	4	toBe
0	5	0	toBe
-10	-2	5	toBe
-7	2	-3.5	toBe
10	3	≈ 3.333	toBeCloseTo
10	0	throw "Division par zéro impossible."	toThrow

Cas de coercion JS à documenter

La classe ne valide pas les types — c'est voulu. Ces tests documentent le comportement **natif de JavaScript** lorsqu'on passe des valeurs inattendues :

Appel	Résultat attendu	Matcher	Ce que ça illustre
add(null, 2)	2	toBe(2)	null est coercé en 0
subtract(undefined, 5)	NaN	toBeNaN()	undefined produit NaN
multiply("abc", 3)	NaN	toBeNaN()	chaîne non numérique → NaN
divide(NaN, 5)	NaN	toBeNaN()	NaN se propage

Ces tests passent sans modifier la classe. Leur rôle est de **rendre visible** la coercion de types JS, souvent source de bugs silencieux en production.

Vérification

```
npm run test:unit
```

Tous les tests doivent être verts. Si un test échoue, corrigez `calculator.js`.

Étape 4 — Le serveur HTTP : `src/server.js`

Architecture

Le serveur utilise uniquement les modules natifs `http` et `url`. Il délègue tout le calcul à la classe `Calculator`.

```
Requête HTTP → server.js (routing + validation) →  
Calculator (calcul) → Réponse JSON
```

Endpoint unique

```
GET /calculate?operation=<op>&a=<nombre>&b=<nombre>
```

Format des réponses

Succès (200) :

```
{  
  "operation": "add",  
  "a": 5,  
  "b": 3,  
  "result": 8  
}
```

Erreur (400 / 404 / 405) :

```
{  
  "error": "message d'erreur"  
}
```

Headers obligatoires sur **toutes** les réponses

```
Content-Type: application/json; charset=utf-8  
Access-Control-Allow-Origin: *  
Access-Control-Allow-Methods: GET, OPTIONS  
Access-Control-Allow-Headers: Content-Type,  
Authorization
```

Tableau des codes HTTP

Situation	Code
Calcul réussi	200
Paramètre manquant	400
Paramètre non numérique	400
Opération inconnue	400

Situation	Code
Division par zéro	400
Route autre que /calculate	404
Méthode autre que GET/OPTIONS	405 + header Allow: GET, OPTIONS
Requête OPTIONS (preflight CORS)	204 sans body

Messages d'erreur exacts

Ces chaînes seront vérifiées mot pour mot par les tests :

Cas	Message
Paramètre manquant	"Paramètres attendus : operation, a, b"
Valeur non numérique	"Les paramètres a et b doivent être des nombres."
Opération inconnue	"Opération inconnue. Utiliser : add, subtract, multiply, divide"
Division par zéro	"Division par zéro impossible." <i>(relayé depuis Calculator)</i>
Route introuvable	"Route introuvable."
Méthode non autorisée	"Méthode non autorisée. Utiliser GET."

Contraintes d'implémentation

1. La fonction `requestHandler` doit être exportée pour que les tests puissent l'importer sans démarrer un vrai serveur :

```
module.exports = { requestHandler, server };
```

2. Le serveur ne démarre que si le fichier est exécuté directement, pas lors d'un `require` :

```
if (require.main === module) {  
  server.listen(PORT, () => {  
    console.log(`Serveur démarré sur  
http://localhost:${PORT}`);  
  });  
}
```

Sans cette condition, chaque `require("../src/server")` dans les tests démarrerait un vrai serveur sur le port 3000 — conflit garanti en CI.

Logique de traitement à implémenter

Voici l'ordre de vérification que doit effectuer `requestHandler` :

1. Positionner les headers CORS sur toutes les réponses
2. Si méthode `OPTIONS` → répondre 204 vide et stopper
3. Si méthode `≠ GET` → répondre 405 avec header `Allow`
4. Si `pathname ≠ /calculate` → répondre 404
5. Si `operation`, `a` ou `b` sont absents de la query string → répondre 400
6. Convertir `a` et `b` en `Number` ; si `NaN` → répondre 400

7. Exécuter l'opération via `Calculator` dans un `try/catch` (pour attraper `divide(x, 0)`)
8. Si opération inconnue → répondre `400`
9. Répondre `200` avec `{ operation, a, b, result }`

Vérification manuelle

```
npm start
# dans un autre terminal :
curl "http://localhost:3000/calculate?
operation=add&a=5&b=3"
curl "http://localhost:3000/calculate?
operation=divide&a=10&b=0"
curl "http://localhost:3000/calculate?
operation=add&a=abc&b=3"
curl -X POST "http://localhost:3000/calculate"
curl "http://localhost:3000/unknown"
```

Étape 5 — L'utilitaire de test HTTP

Pour les tests d'intégration, vous avez besoin d'un helper qui envoie

de vraies requêtes HTTP à votre serveur sans passer par le réseau public.

Copiez ce code dans `tests/helpers/http.js` :

```
const http = require("http");

/**
```

* Envoie une requête HTTP à une instance de serveur de test.

* @param {http.Server} server - Instance déjà en écoute

* @param {string} path - Chemin + query string

* @param {string} method - Méthode HTTP (défaut : "GET")

* @returns {Promise<{status, headers, body, duration}>}

*/

```
function request(server, path, method = "GET") {
  return new Promise((resolve, reject) => {
    const addr = server.address();
    const start = Date.now();
    const req = http.request(
      { hostname: "127.0.0.1", port: addr.port,
path, method },
      (res) => {
        let data = "";
        res.on("data", (chunk) => { data +=
chunk; });
        res.on("end", () => resolve({
          status: res.statusCode,
          headers: res.headers,
          body: data ? JSON.parse(data)
: null,
          duration: Date.now() - start,
        }));
      }
    );
    req.on("error", reject);
    req.end();
  });
}
```

```
    });  
  }  
  
  module.exports = { request };
```

Port aléatoire `listen(0)` : dans les tests, vous démarrerez le serveur avec le port `0`.

Le système d'exploitation assigne alors un port libre automatiquement — indispensable pour éviter les conflits entre tests parallèles dans GitHub Actions.

Étape 6 — Tests d'intégration : `tests/api.test.js`

Les tests d'intégration testent l'API comme un **client externe** : vraie requête HTTP, vraie réponse JSON, vrais codes de statut.

Structure de base

```
const http = require("http");  
const { requestHandler } = require("../src/server");  
const { request } = require("../helpers/http");  
  
describe("API /calculate", () => {  
  let server;  
  
  beforeAll((done) => {  
    server = http.createServer(requestHandler);
```

```
        server.listen(0, "127.0.0.1", done); // port
aléatoire
    });

    afterAll((done) => {
        server.close(done);
    });

    // vos tests ici
});
```

Tests à écrire — organisés par blocs `describe`

`describe("Performance")`

- Une requête valide répond en moins de 100 ms
- Une requête en erreur 400 répond en moins de 100 ms

```
const { duration } = await request(server,
"/calculate?operation=add&a=1&b=2");
expect(duration).toBeLessThan(100);
```

`describe("Headers de réponse")`

Vérifiez sur une réponse 200, une 400 et une 404 que :

- `Content-Type` vaut **exactement** `"application/json; charset=utf-8"` (assertion stricte `.toBe`)
- `Access-Control-Allow-Origin` vaut `"*"`

describe("OPTIONS /calculate – preflight CORS")

- Status 204
- Body null
- Access-Control-Allow-Origin vaut "*"
- Access-Control-Allow-Methods contient "GET"

```
const { status, body } = await request(server,  
  "/calculate", "OPTIONS");  
expect(status).toBe(204);  
expect(body).toBeNull();
```

describe("GET /calculate – cas nominaux")

Utilisez `it.each` pour les 8 combinaisons :

operation	a	b	expected
add	2	3	5
subtract	10	4	6
multiply	6	7	42
divide	20	5	4
add	-5	-3	-8
subtract	-5	-3	-2
multiply	-3	-4	12
divide	-10	-2	5

Chaque test vérifie `status === 200` et le body avec `toMatchObject` :

```
expect(body).toMatchObject({ operation, a, b, result:
  expected });
```

Tests nominaux supplémentaires à ajouter dans ce bloc :

Description	URL	Vérification
Division décimale	<code>divide&a=10&b=3</code>	<code>body.result</code> proche de 3.333 avec <code>toBeCloseTo</code>
Décimaux en query string	<code>add&a=1.5&b=2.5</code>	<code>body.result === 4</code>
Contrat JSON 200	<code>multiply&a=3&b=4</code>	body contient <code>operation, a, b, result</code> et pas <code>error</code>

```
describe("Méthode non autorisée")
```

Méthode	Vérification
POST	Status 405 , body a <code>error</code>
POST	Header <code>allow</code> contient <code>"GET"</code>
PUT	Status 405

```
describe("GET /calculate – erreurs 400")
```

Cas	URL	Vérification
b manquant	?operation=add&a=2	status 400 , message /Paramètres attendus/
a manquant	?operation=add&b=2	status 400 , message /Paramètres attendus/
a non numérique	?operation=add&a=abc&b=3	status 400 , message /doivent être des nombres/
b non numérique	?operation=add&a=3&b=abc	status 400 , message /doivent être des nombres/
Division par zéro	?operation=divide&a=10&b=0	status 400 , message exact "Division par zéro impossible."
Opération inconnue	?operation=modulo&a=10&b=3	status 400 , message /Opération inconnue/
operation absent	?a=5&b=3	status 400 , message /Paramètres attendus/

Cas	URL	Vérification
Contrat JSON erreur	?operation=add&a=2	body a error et pas result

describe("GET – autres routes")

Cas	URL	Vérification
Route inconnue	/unknown	status 404 , body.error === "Route introuvable."
Racine	/	status 404 , body a error
Slash final	/calculate/	status 404 , body a error

describe("Cas limites – edge cases")

Cas	URL	Vérification
Très grande valeur	add&a=1e308&b=1e308	status 200 , body.result est null , "Infinity" ou Infinity
a=-0	add&a=-0&b=5	status 200 , body.result === 5 , body.a === 0

JSON.stringify(-0) produit "0" — ce test documente que le client reçoit 0 ,
pas -0 , même si JavaScript distingue les deux en interne.

Vérification

```
npm run test:integration
```

Étape 7 — Couverture de code

```
npm run test:coverage
```

Jest affiche un tableau dans le terminal. Objectif :

```
-----|-----|-----|-----|-----
--|
File          | % Stmts | % Branch | % Funcs | %
Lines |
-----|-----|-----|-----|-----
--|
  calculator.js |    100   |    100    |    100   |    100
|
  server.js     |   ≥ 90   |   ≥ 90    |   ≥ 90   |   ≥ 90
|
-----|-----|-----|-----|-----
--|
```

Si Jest échoue avec `Coverage threshold not met`, la colonne `Uncovered Line #s`

indique les lignes jamais exécutées — ajoutez les tests manquants.

Un rapport HTML est généré dans `coverage/index.html` :
ouvrez-le dans le navigateur
pour visualiser les branches non couvertes ligne par ligne.

Étape 8 — GitHub Actions : `.github/workflows/ci.yml`

Objectif

À chaque `git push`, GitHub exécute automatiquement :

1. **Lint** — ESLint vérifie la qualité du code
2. **Tests** — Jest s'exécute sur Node.js 22 et Node.js 24 en parallèle

Le fichier `ci.yml` à créer

```
name: CI – Tests & Couverture

on:
  push:
    branches: ["**"]           # tous les push, toutes
    branches
  pull_request:
    branches: ["main"]

env:
  FORCE_JAVASCRIPT_ACTIONS_TO_NODE24: true

jobs:
```

lint:

name: Lint (ESLint)

runs-on: ubuntu-latest

steps:

- uses: actions/checkout@v5
- uses: actions/setup-node@v4
with:
node-version: "22.x"
- run: npm install
- run: npm run lint

test:

name: Tests Node.js \${{ matrix.node-version }}

runs-on: ubuntu-latest

needs: lint # attend que le

lint réussisse

strategy:

fail-fast: false # teste les deux

versions même si l'une échoue

matrix:

node-version: ["22.x", "24.x"]

steps:

- uses: actions/checkout@v5
- uses: actions/setup-node@v4
with:
node-version: \${{ matrix.node-version }}
- run: npm install
- run: npm run test:ci
- name: Upload du rapport de couverture
if: matrix.node-version == '22.x'

```
uses: actions/upload-artifact@v4
with:
  name: coverage-report
  path: coverage/
  retention-days: 30
```

Explication des concepts clés

Concept	Explication
<code>on: push: branches: ["**"]</code>	Déclenche le CI sur n'importe quel push
<code>needs: lint</code>	Le job <code>test</code> ne démarre qu'après le succès de <code>lint</code>
<code>matrix: node-version</code>	Répète le job pour chaque version de Node listée
<code>fail-fast: false</code>	Les deux matrices s'exécutent indépendamment
<code>upload-artifact</code>	Sauvegarde le rapport HTML de couverture 30 jours

Pousser et observer

```
git add .
git commit -m "feat: API calculatrice avec tests Jest et CI GitHub Actions"
git push origin main
```

Ouvrez l'onglet **Actions** de votre dépôt GitHub.

Vous devez voir 3 jobs s'exécuter et se terminer en vert :

- ✓ Lint (ESLint)
- ✓ Tests Node.js 22.x
- ✓ Tests Node.js 24.x

Critères de validation

- ☐ `npm run lint` → aucune erreur
- ☐ `npm test` → tous les tests passent
- ☐ `npm run test:coverage` → $\geq 90\%$ sur toutes les métriques
- ☐ Pipeline GitHub Actions → 100% vert sur Node 22 et Node 24

Référence rapide des commandes

```
npm run lint           # vérification ESLint
npm test              # tous les tests
npm run test:unit      # tests unitaires seuls
npm run test:integration # tests d'intégration seuls
npm run test:coverage  # tests + rapport de
couverture
npx jest --watch       # mode watch (relance à
chaque sauvegarde)
npx jest --testNamePattern="add" # filtrer par nom
de test
```